Incremental timing update

Real timers cannot rebuild the chip after every buffer insert

Goldens to remember

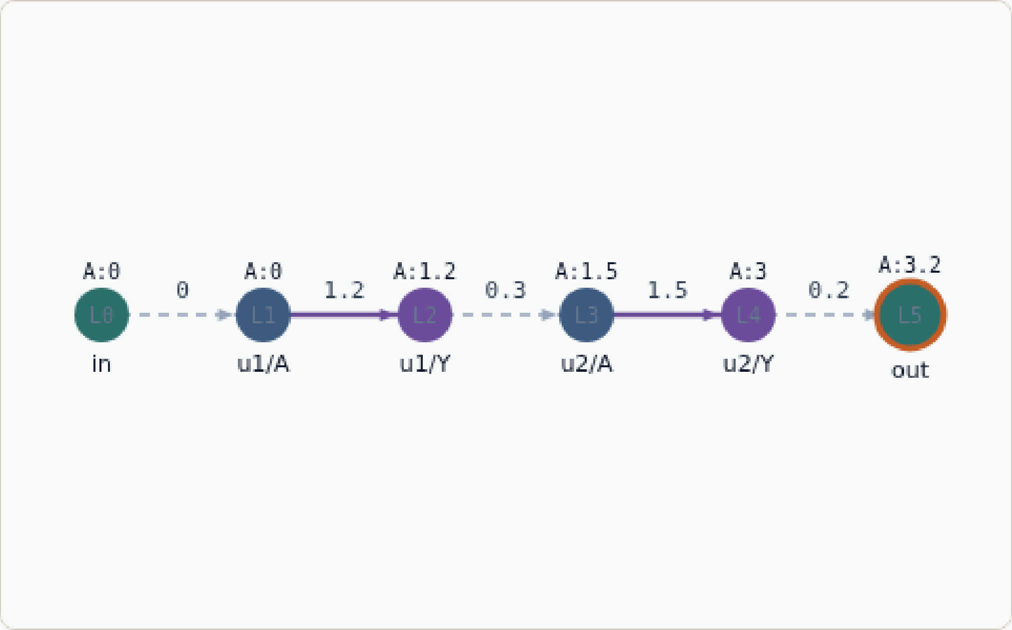
- Invalidated cone: u1/Y, u2/A, u2/Y, out
- Clean: in, u1/A
- ΔA(out)=+0.8
- Keep these numbers handy, the browser challenges and Track A tests use the same instance
- <!-- algorithm-walkthrough -->

Start from a full analysis

INCREMENTAL TIMING UPDATE

Start from a full analysis

Step 1 / 5 · base-tags · module03-01-incremental-update



Explanation

Base arrival at out is 3.2 with setup slack 6.8. Incremental update never starts from an empty graph—it starts from valid tags.

- Full propagate first
- A(out)=3.2
- setup slack=6.8

```
A(out)=3.2
S(out)=6.8
```

Start from a full analysis

Edit one cell delay

INCREMENTAL TIMING UPDATE

Edit one cell delay

Step 2 / 5 · local-edit · module03-01-incremental-update

Explanation

Bump the u1 cell arc from 1.2 to 2.0. Only the fanout cone of u1/Y can change—everything upstream of the edit stays put.

- Edit u1/A → u1/Y delay
- 1.2 → 2.0
- Upstream tags stay

```
edit arc: u1/A|u1/Y
new delay: 2.0
```

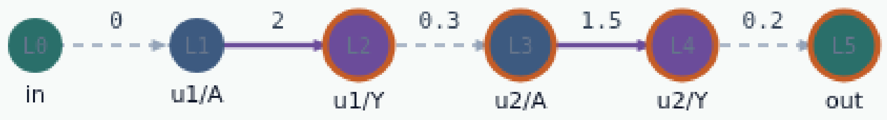
Edit one cell delay

Invalidate the fanout cone

INCREMENTAL TIMING UPDATE

Invalidate the fanout cone

Step 3 / 5 · invalidate-cone · module03-01-incremental-update



Explanation

Mark u1/Y, u2/A, u2/Y, and out dirty. Do not touch in or u1/A—their arrivals are still valid.

- BFS successors from edit.to
- Delete dirty arrivals
- Keep clean tags

```
invalidated: u1/Y, u2/A, u2/Y, out
clean: in, u1/A
```

Invalidate the fanout cone

Recompute only the dirty pins

INCREMENTAL TIMING UPDATE

Recompute only the dirty pins

Step 4 / 5 · recompute · module03-01-incremental-update

Explanation

Replay topo order on dirty pins. Arrival at out becomes 4.0; setup slack drops to 6.0. Same answer as a full rebuild, less work.

- Recompute dirty only
- A(out)=4.0
- setup slack=6.0

```
A(out)=4.0
S(out)=6.0
ΔA = +0.8
```

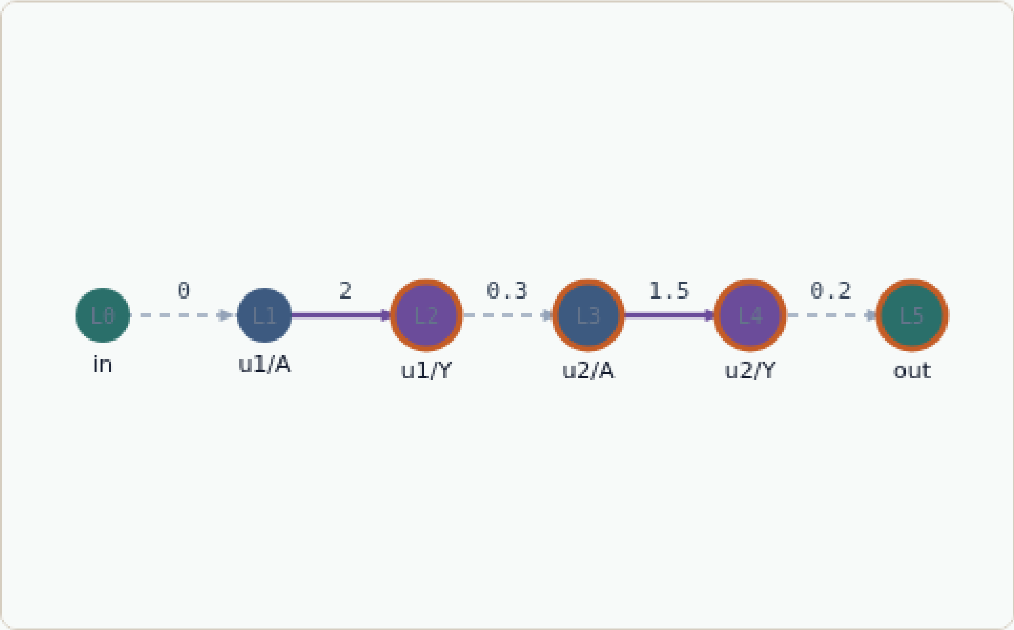
Recompute only the dirty pins

Why timers insist on incremental

INCREMENTAL TIMING UPDATE

Why timers insist on incremental

Step 5 / 5 · why-incremental · module03-01-incremental-update



Explanation

Place, CTS, and ECO change tiny regions millions of times. Full-chip rebuilds every edit would not finish. Cone invalidate-and-repair is the habit.

- Edits are local
- Cones stay small
- Correctness = same as full rebuild

```
cone size: 4 pins
full graph: 6 pins
```

Why timers insist on incremental

Browser lab track

- In the browser lab, open **incremental-update**
- Load the starter, run the analysis once, and read the metrics panel
- Orient yourself, challenge panel, canvas, Check, then mirror the same goldens in code

Implement track

- In the implement track
- Run `python3 common/test_propagate.py` (and the timing-graph test) until the goldens print

Pitfall

- Do not mix setup and hold required maps
- Do not propagate before the graph is levelized
- After an edit or exception, recompute, stale tags lie

Your turn

- Finish the checklist on at least one track, preferably both
- When your numbers match the goldens, take the quiz, then continue